الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Guide to 80x86 Assembly Programming: Instructions, Flags, and System Control

Flag Control Instructions

- **LAHF** and **SAHF** explanations: Correctly describe the operations for loading and storing the AH register with flag values.
- **CLC, STC, CLI, and STI:** These instructions are clearly explained with their effects on specific processor flags (Carry Flag or Interrupt Flag).
- **Example:**

```
LAHF
MOV [SI], AH
MOV AH, [DI]
SAHF
```

Comparison Instructions (CMP)

The ***CMP*** instruction compares two operands by performing subtraction and updating flags without storing the result.

Format:

```assembly
CMP destination, source
```

- Updates the flags: CF, AF, OF, PF, SF, ZF.
- Possible operand combinations:
  - Register-to-register
  - Memory-to-register
  - Immediate-to-register/memory

Example:

```assembly
MOV AL, 99H  ; Load AL with 99 (hex)
CMP AL, 1BH  ; Compare AL with 27 (hex)
```

After execution, flags like CF and ZF determine the relation between operands.

Jump Instructions

Jump instructions alter the flow of execution based on flag conditions.

Unconditional Jump (JMP)

Jumps to the specified address unconditionally.

```assembly
JMP 1234H  ; Jump to address 1234H
```

Conditional Jumps

These depend on specific flag values. For instance:

- `JZ` (Jump if Zero): Executes if `ZF=1`.
- `JC` (Jump if Carry): Executes if `CF=1`.
- `JA` (Jump if Above): Executes if `CF=0` and `ZF=0`.

Example of Conditional Jumps:

```assembly
MOV AX, 5
CMP AX, 3
JA Label  ; Jump if AX > 3
Label:
```

Stack Operations

The stack provides temporary storage for data during program execution. The PUSH and POP instructions manage stack content.

Format:

- PUSH Source: Decreases the stack pointer (SP) by 2 and stores the value.
- POP Destination: Retrieves the value from the stack and increases SP by 2.

Example:

```assembly
PUSH AX  ; Save AX to stack
POP BX   ; Load the saved value into BX
```

Subroutine Handling

A subroutine is a reusable block of code that can be called from different program sections.

Key Instructions:

- CALL: Transfers execution to the subroutine.
- RET: Returns to the caller.

Example:

```assembly
CALL Subroutine
Subroutine:
    MOV AX, 1
    RET
```

Nested Subroutines:
Allows one subroutine to call another.

```assembly
Main:
    CALL Sub1
    ...
Sub1:
    CALL Sub2
    RET
Sub2:
    RET
```

Loop Instructions
Loops repeat code execution based on the CX register value.

Types:

- **LOOP**: Decrements CX and repeats if CX ≠ 0.
- **LOOPE**: Repeats while CX ≠ 0 and ZF = 1.
- **LOOPNE**: Repeats while CX ≠ 0 and ZF = 0.

Example:

```assembly
MOV CX, 5
Label:
    ; Code to repeat
    LOOP Label
```

Interrupts

Interrupts pause the current task and execute an Interrupt Service Routine (ISR).

Categories:

1. **Processor interrupts: Triggered by the CPU.**
2. **Software interrupts: Explicitly invoked using the INT instruction.**
3. **Hardware interrupts: Initiated by external devices.**

Example:

INT 21H provides DOS services:

- Function 09: Display a string.

```assembly
MOV AH, 09H
MOV DX, OFFSET Message
INT 21H
Message DB 'Hello, World$', 0
```

INT 10H handles video operations:

- Function 02: Set cursor position.

```assembly
MOV AH, 02H
MOV BH, 0
MOV DH, 12 ; Row
MOV DL, 39 ; Column
INT 10H
```

Conclusion:

In conclusion, this document provides a detailed exploration of essential 80x86 assembly programming concepts, covering instruction sets, flag operations, and system control. It emphasizes practical examples to illustrate the usage of comparison, jump, stack, and loop instructions. Key aspects of interrupt handling and subroutine management are also discussed, showcasing their importance in efficient programming. This guide serves as a foundational resource for understanding low-level programming and microprocessor operations. With a structured approach, it bridges theoretical knowledge and practical application, making it invaluable for students and professionals alike.